MENDSPEECH REVISED LEARNING SYSTEM

Week 4: FastConformer and Efficient Encoder Behavior

Measure why FastConformer is efficient and freeze a reproducible baseline.

> Six focused build days plus one consolidation day. Each day ends with named artifacts or measured evidence.

Week milestone

> Measure why FastConformer is efficient and freeze a reproducible baseline.

Week map

Day	Focus	Minimum evidence	Compute
Day 22	Why FastConformer exists	• Estimate attention matrix size before and after aggressive temporal subsampling.	Local CPU
Day 23	Temporal subsampling experiment	• Compare 2x, 4x, and 8x temporal reduction on tensor length, runtime, and rough output behavior.	Modal L4 useful
Day 24	Pretrained FastConformer baseline	• Benchmark WER, latency, and GPU memory by damage type.	Modal L4
Day 25	Context and attention limits	• If supported, compare at least two context settings on the same subset.	Modal L4
Day 26	Efficiency benchmark harness	• Run repeated inference and calculate variance. • Detect and discard obviously invalid cold start comparisons.	Modal L4
Day 27	FastConformer failure casebook	• Look for systematic error patterns rather than isolated anecdotes.	Modal L4
Day 28	Week 4 integration	• Run the same ten reference clips through the full Week 2 uncertainty policy using FastConformer.	Modal L4

Reference spine

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 22

Why FastConformer exists

Compute
Local CPU

LEARN

- Sequence length as an attention cost driver.
- Subsampling before expensive encoder blocks.
- Depthwise separable convolution.
- Local and limited context attention.

BUILD IN MENDSPEECH

- Read the FastConformer paper with a comparison checklist.
- Write a diagram showing what changes relative to Conformer.

EXPERIMENT AND MEASURE

- Estimate attention matrix size before and after aggressive temporal subsampling.

REQUIRED OUTPUT

- docs/day22_fastconformer_notes.md
- results/day22_compute_estimates.csv

> **Completion check:** You can explain FastConformer as a set of concrete efficiency choices, not just a faster model name.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 23

Temporal subsampling experiment

Compute
Modal L4 useful

LEARN

- Convolutional subsampling.
- Temporal resolution.
- Information loss versus compute reduction.

BUILD IN MENDSPEECH

- Implement a small subsampling front end or isolate one from a framework.
- Track frames per second before and after each stage.

EXPERIMENT AND MEASURE

- Compare 2x, 4x, and 8x temporal reduction on tensor length, runtime, and rough output behavior.

REQUIRED OUTPUT

- src/models/subsampling.py
- results/day23_subsampling.csv

> **Completion check:** You can quantify how subsampling changes sequence length and downstream attention cost.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 24

Pretrained FastConformer baseline

Compute
Modal L4

LEARN

- Model checkpoint loading.
- Tokenizer and decoder configuration.
- Batch versus single utterance inference.

BUILD IN MENDSPEECH

- Run a current NeMo FastConformer checkpoint on your clean and damaged sets.
- Record model revision and all inference settings.

EXPERIMENT AND MEASURE

- Benchmark WER, latency, and GPU memory by damage type.

REQUIRED OUTPUT

- src/asr/fastconformer_runner.py
- results/day24_fastconformer_baseline.csv
- configs/model_baseline.yaml

> **Completion check:** You have a reproducible baseline with model, data, hardware, and settings fixed.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 25

Context and attention limits

Compute
Modal L4

LEARN

- Full context attention.
- Limited context attention.
- Left and right context.
- Accuracy versus latency intuition.

BUILD IN MENDSPEECH

- Inspect context settings in the model configuration.
- Create a visual timeline explaining visible past and future context.

EXPERIMENT AND MEASURE

- If supported, compare at least two context settings on the same subset.

REQUIRED OUTPUT

- docs/day25_context_timeline.md
- results/day25_context_compare.csv

> **Completion check:** You can explain exactly why future context creates algorithmic latency.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 26

Efficiency benchmark harness

Compute
Modal L4

LEARN

- Warmup runs.
- Synchronized GPU timing.
- Median and percentile latency.
- Real time factor.
- Peak memory.

BUILD IN MENDSPEECH

- Create one benchmark function used by every later experiment.
- Log environment and model metadata automatically.

EXPERIMENT AND MEASURE

- Run repeated inference and calculate variance.
- Detect and discard obviously invalid cold start comparisons.

REQUIRED OUTPUT

- src/bench/benchmark_asr.py
- src/bench/environment.py
- results/day26_repeatability.csv

> **Completion check:** Repeated runs produce stable enough numbers to support comparisons.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 27

FastConformer failure casebook

Compute
Modal L4

LEARN

- Error slicing by corruption type and severity.
- Short versus long utterance effects.
- Confidence versus error.

BUILD IN MENDSPEECH

- Build a casebook of at least fifteen interesting failures.
- Link each case to audio, transcript, confidence, and damage metadata.

EXPERIMENT AND MEASURE

- Look for systematic error patterns rather than isolated anecdotes.

REQUIRED OUTPUT

- results/fastconformer_failure_casebook.md

> **Completion check:** You can name at least three repeatable failure patterns and propose a testable reason for each.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 28

Week 4 integration

Compute
Modal L4

LEARN

- Review efficiency choices and baseline results.

BUILD IN MENDSPEECH

- Replace the generic ASR runner in MendSpeech with the reproducible FastConformer path.
- Expose latency, RTF, WER when reference text exists, and GPU memory in the research console.

EXPERIMENT AND MEASURE

- Run the same ten reference clips through the full Week 2 uncertainty policy using FastConformer.

REQUIRED OUTPUT

- app/mendspeech_v1_fastconformer.py
- reports/week4_fastconformer.md

> **Completion check:** MendSpeech now has a measured, inspectable FastConformer recognition core.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation